In [5]:
```
import numpy as np
import pandas as pd
```

In [6]:
```
from sklearn.datasets import load_iris

iris=load_iris()

df=pd.DataFrame(iris.data,columns=['sepal.length','sepal.width','petal.length','pet

df['variety']=iris.target_names[iris.target]
```

In [7]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

In [8]:
```
df.variety.value_counts()
```

Out[8]:
```
variety
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

In [9]:
```
df.head()
```

Out[9]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

In [10]:
```
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values
```

In [11]:
```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

In [12]:
```
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_state
```

In [13]:
```
model_KNN=KNeighborsClassifier(n_neighbors=5)
```

In [14]:
```
model_KNN.fit(xtrain,ytrain)
```

Out[14]:
```
▾ KNeighborsClassifier ⓘ ?

▸ Parameters

▸ Fitted attributes
```

In [15]:
```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
```

```
0.95
0.9666666666666667
```

In [16]:
```
from sklearn.metrics import confusion_matrix
```

In [17]:
```
confusion_matrix(label,model_KNN.predict(features))
```

Out[17]:
```
array([[50,  0,  0],
       [ 0, 45,  5],
       [ 0,  2, 48]])
```

In [18]:
```
from sklearn.metrics import classification_report
```

In [19]:
```
print(classification_report(label,model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        50
  versicolor       0.96      0.90      0.93        50
   virginica       0.91      0.96      0.93        50

    accuracy                           0.95       150
   macro avg       0.95      0.95      0.95       150
weighted avg       0.95      0.95      0.95       150
```

In [ ]: